



VineCare

AWS Cloud Architecture

Technical Documentation

Region: Europe (Frankfurt) — eu-central-1

1. Executive Summary

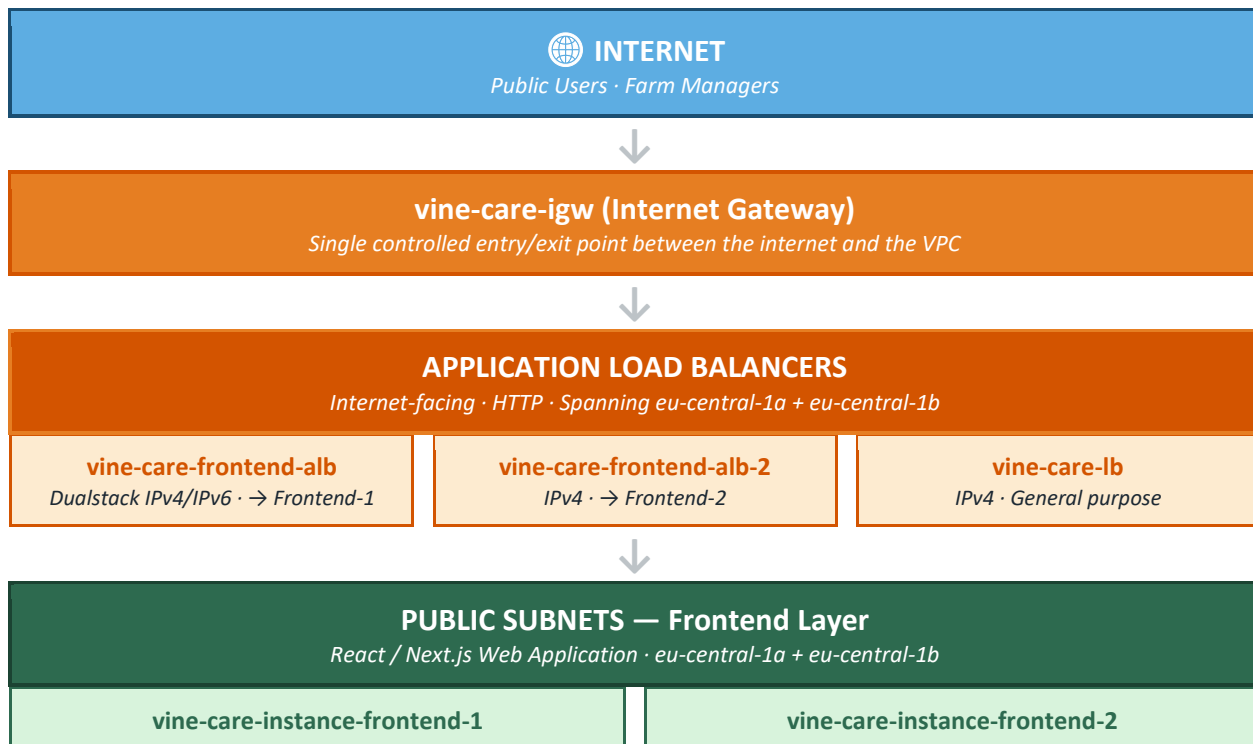
VineCare is a precision viticulture platform that empowers vineyard managers with drone imagery analysis, phenology tracking, agronomic KPIs, and AI-driven yield predictions. The platform is fully deployed on Amazon Web Services (AWS) in the eu-central-1 (Frankfurt) region, leveraging two Availability Zones for high availability and fault tolerance.

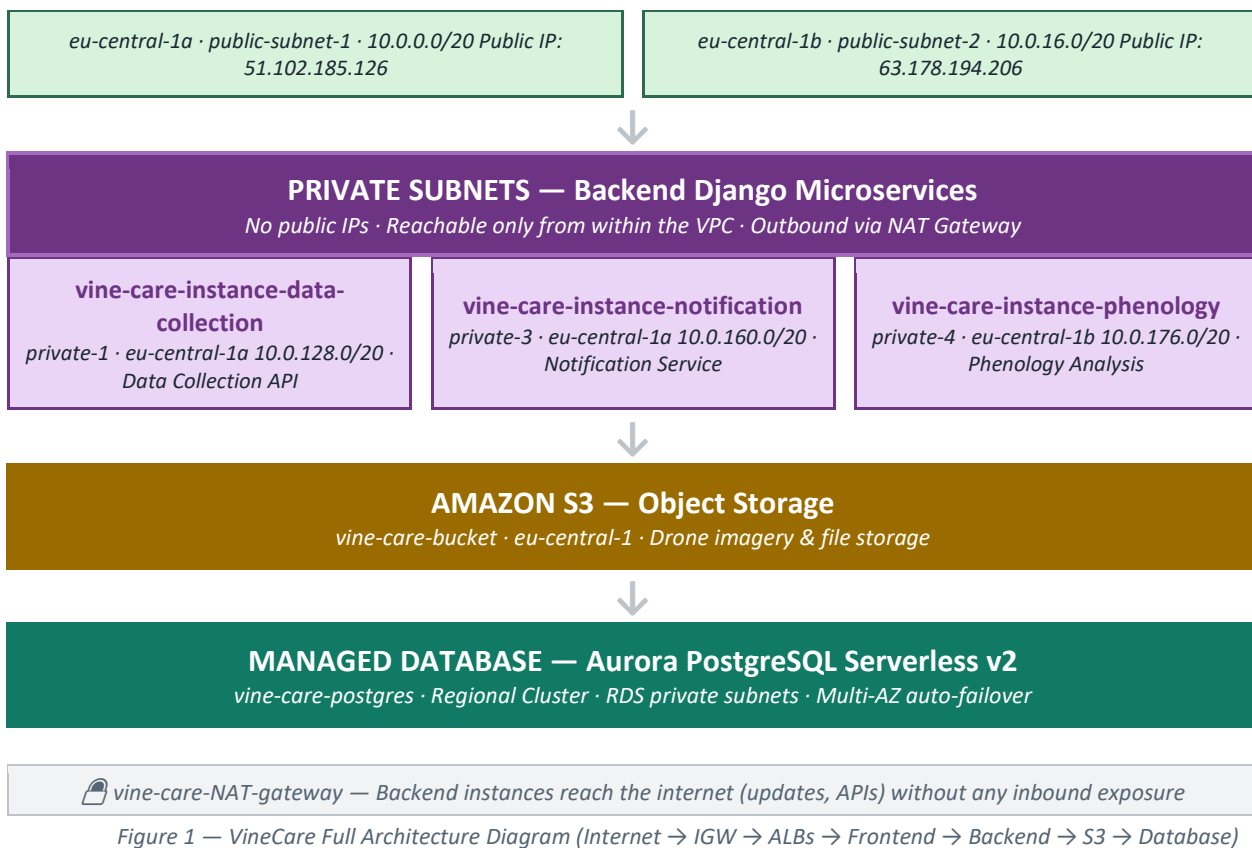
The architecture cleanly separates public-facing frontend services from privately isolated backend Django microservices, with Amazon S3 for scalable image storage and a fully managed Aurora PostgreSQL Serverless v2 database that auto-scales with workload demand.

EC2 Instances	Load Balancers	Availability Zones	S3 Buckets	Database
5 × t3.micro	3 × App LB	2 (1a + 1b)	1 (vine-care-bucket)	Aurora Serverless v2

2. Architecture Overview

The diagram below shows the complete VineCare infrastructure — from the public internet through load balancers, frontend instances, backend microservices, S3 object storage, and the managed database. All layers are contained within a single dedicated VPC.





3. End-to-End Request Flow

The diagram below shows exactly how a request from a farm manager travels through each layer of the VineCare infrastructure, from their browser to the database and back.

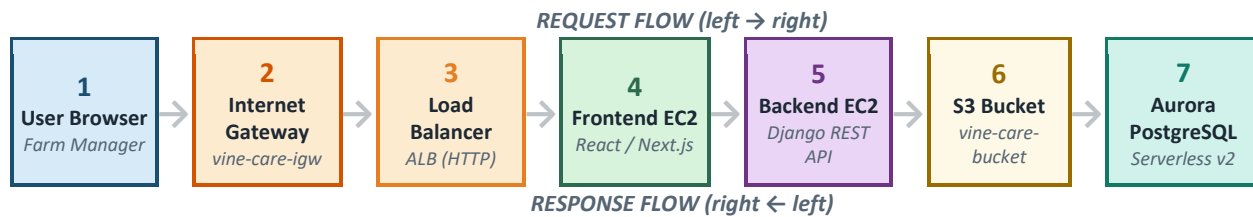


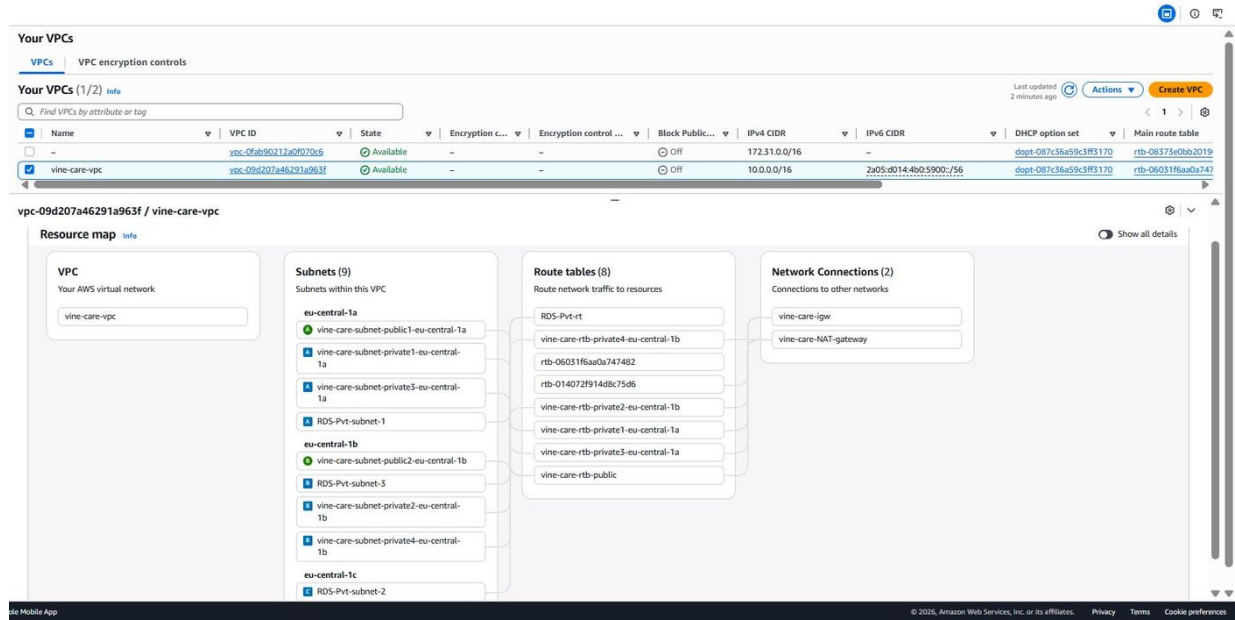
Figure 2 — End-to-End Request & Response Flow Diagram

#	Layer	What Happens
1	User Browser	Farm manager opens the VineCare web app. DNS resolves to the ALB DNS name.
2	Internet Gateway	Traffic enters the VPC through vine-care-igw — the single controlled entry/exit point for internet traffic.
3	Load Balancer	The Application Load Balancer terminates TLS/HTTP and routes the request to a healthy frontend EC2 instance.
4	Frontend EC2	The React/Next.js application renders the UI and issues REST API calls to the Django backend microservices.
5	Backend EC2	The appropriate Django microservice (data-collection, notification, or phenology) processes the request.
6	S3 Bucket	Drone images uploaded by field operators are stored in vine-care-bucket. The data-collection service writes/reads image files from S3.
7	Aurora PostgreSQL	The database executes queries. Aurora Serverless v2 scales compute capacity automatically if load increases.

Return path: The response travels back in reverse. Backend instances reach external services (updates, APIs) via vine-care-NAT-gateway — no inbound connections to private subnets are possible from the internet.

4. VPC & Network Design

All VineCare resources reside inside vine-care-vpc with CIDR 10.0.0.0/16. Resources are distributed across two Availability Zones (eu-central-1a and eu-central-1b) for redundancy. Below is the AWS Console view of the VPC resource map.



Screenshot 1 — AWS VPC Console: vine-care-vpc Resource Map

4.1 Subnet Layout Diagram

The diagram below maps each subnet to its CIDR, availability zone, and the service running within it:

vine-care-vpc · 10.0.0.0/16 · eu-central-1 (Frankfurt)

Availability Zone: eu-central-1a	Availability Zone: eu-central-1b
vine-care-subnet-public1-eu-central-1a CIDR: 10.0.0.0/20 → Frontend-1 EC2 · 51.102.185.126	vine-care-subnet-public2-eu-central-1b CIDR: 10.0.16.0/20 → Frontend-2 EC2 · 63.178.194.206
vine-care-subnet-private1-eu-central-1a CIDR: 10.0.128.0/20 → data-collection Django service	vine-care-subnet-private4-eu-central-1b CIDR: 10.0.176.0/20 → phenology Django service
vine-care-subnet-private3-eu-central-1a CIDR: 10.0.160.0/20 → notification Django service	RDS-Pvt-subnet-2 / RDS-Pvt-subnet-3 CIDR: 10.0.32.128/25 · 10.0.33.0/25 → Aurora PostgreSQL (DB only)
RDS-Pvt-subnet-1	

CIDR: 10.0.32.0/25
→ Aurora PostgreSQL (DB only)

Figure 3 — VPC Subnet Layout across both Availability Zones

4.2 Subnet Reference Table

All active subnets from the AWS Subnets console are listed below with their exact CIDRs (IP Ranges):

Name	Subnet ID	State	VPC	Block Public	IPv4 CIDR	IPv6 CIDR	IPv6 CIDR
RDS-Pvt-subnet-2	subnet-05429c1844147bccc	Available	vpc-09d207a46291a963f vine...	Off	10.0.32.128/25	--	--
vine-care-subnet-private1-eu-central-1a	subnet-068d65125452ab119	Available	vpc-09d207a46291a963f vine...	Off	10.0.128.0/20	--	--
RDS-Pvt-subnet-3	subnet-077172300de9821ad	Available	vpc-09d207a46291a963f vine...	Off	10.0.33.0/25	--	--
vine-care-subnet-public1-eu-central-1a	subnet-0f1154e9c7b1b01a7	Available	vpc-09d207a46291a963f vine...	Off	10.0.0.0/20	2a05:d014:4b0:5900::/64	subnet-...
vine-care-subnet-private3-eu-central-1a	subnet-016104b6230c16c8f	Available	vpc-09d207a46291a963f vine...	Off	10.0.160.0/20	--	--
vine-care-subnet-private2-eu-central-1b	subnet-02aa143c91fa9f8f3	Available	vpc-09d207a46291a963f vine...	Off	10.0.144.0/20	--	--
vine-care-subnet-private4-eu-central-1b	subnet-006c6dc64a41724fb	Available	vpc-09d207a46291a963f vine...	Off	10.0.176.0/20	--	--
RDS-Pvt-subnet-1	subnet-0d382825d461356e3	Available	vpc-09d207a46291a963f vine...	Off	10.0.32.0/25	--	--
vine-care-subnet-public2-eu-central-1b	subnet-0df8f5f25df65dd1	Available	vpc-09d207a46291a963f vine...	Off	10.0.16.0/20	2a05:d014:4b0:5901::/64	subnet-...

Screenshot 2 — AWS VPC Console: All Subnets in vine-care-vpc

Subnet Name	IPv4 CIDR	AZ	Type	Used For
vine-care-subnet-public1-eu-central-1a	10.0.0.0/20	eu-central-1a	Public	Frontend-1 EC2
vine-care-subnet-public2-eu-central-1b	10.0.16.0/20	eu-central-1b	Public	Frontend-2 EC2
vine-care-subnet-private1-eu-central-1a	10.0.128.0/20	eu-central-1a	Private	Data Collection
vine-care-subnet-private3-eu-central-1a	10.0.160.0/20	eu-central-1a	Private	Notification
vine-care-subnet-private4-eu-central-1b	10.0.176.0/20	eu-central-1b	Private	Phenology
RDS-Pvt-subnet-1	10.0.32.0/25	eu-central-1a	Private	Aurora DB only
RDS-Pvt-subnet-2	10.0.32.128/25	eu-central-1b	Private	Aurora DB only
RDS-Pvt-subnet-3	10.0.33.0/25	eu-central-1b	Private	Aurora DB only

Table 1 — Complete Subnet Reference

4.3 Routing & Connectivity

- vine-care-igw (Internet Gateway) — Attached to the VPC. Enables inbound internet traffic to public subnets and outbound traffic from public subnet instances.

- vine-care-NAT-gateway — Deployed in the public subnet. Allows the three private backend EC2 instances to make outbound connections (package updates, external APIs) without being reachable from the internet.
- vine-care-rtb-public — Route table for public subnets. Routes 0.0.0.0/0 to the Internet Gateway.
- vine-care-rtb-private1/3 (eu-central-1a) — Route tables for private subnets in AZ 1a. Outbound traffic routes via NAT Gateway.
- vine-care-rtb-private4 (eu-central-1b) — Route table for the private subnet in AZ 1b. Outbound via NAT Gateway.
- RDS-Pvt-rt — Dedicated route table for RDS subnets, keeping all database traffic isolated from application traffic.

5. Compute Layer — EC2 Instances

VineCare runs five EC2 t3.micro instances, all in Running state with 3/3 health checks passed. Below is the AWS EC2 console view.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...
vine-care-instance-data-collection	i-0d8f2e0b6fe85a56b	Running	t3.micro	3/3 checks passed	View alarms +	eu-central-1a	-	-
vine-care-instance-frontend-1	i-033eb4edae208d4dd	Running	t3.micro	3/3 checks passed	View alarms +	eu-central-1a	ec2-51-102-185-126.eu...	51.102.185.126
vine-care-instance-phenology	i-013bfebbd01bb721	Running	t3.micro	3/3 checks passed	View alarms +	eu-central-1b	-	-
vine-care-instance-frontend-2	i-011e049b04e71bfa2	Running	t3.micro	3/3 checks passed	View alarms +	eu-central-1b	ec2-63-178-194-206.eu...	63.178.194.206
vine-care-instance-notification	i-02d8faa6543512675	Running	t3.micro	3/3 checks passed	View alarms +	eu-central-1a	-	-

Screenshot 3 — AWS EC2 Console: All Five Running Instances

Instance Name	Role	Type	Subnet	AZ	Public IP
vine-care-instance-frontend-1	React/Next.js Frontend	t3.micro	public-1	eu-central-1a	51.102.185.126
vine-care-instance-frontend-2	React/Next.js Frontend	t3.micro	public-2	eu-central-1b	63.178.194.206
vine-care-instance-data-collection	Django – Data Collection API	t3.micro	private-1	eu-central-1a	—
vine-care-instance-notification	Django – Notification Service	t3.micro	private-3	eu-central-1a	—
vine-care-instance-phenology	Django – Phenology Analysis	t3.micro	private-4	eu-central-1b	—

Table 2 — EC2 Instance Inventory with Roles, Subnets, and IPs

5.1 Frontend Instances

Two frontend EC2 instances serve the React/Next.js VineCare web application. Each is placed in a public subnet in a different Availability Zone, providing geographic redundancy. Each instance has a static public IPv4 address and is fronted by its own Application Load Balancer.

5.2 Backend Microservices (Private Instances)

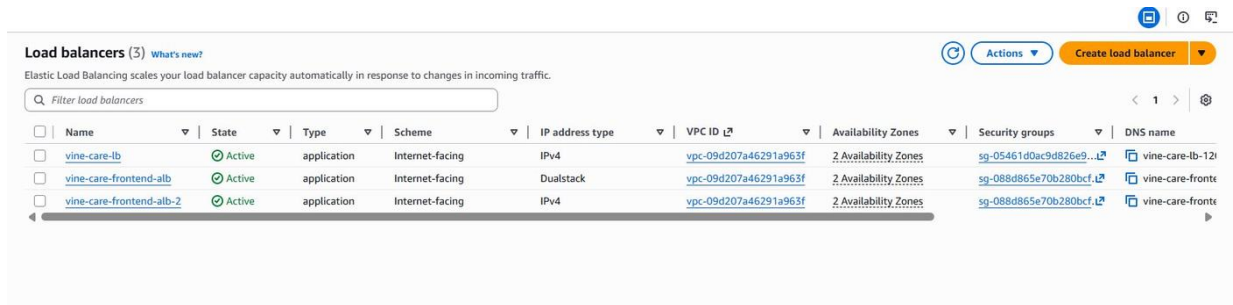
Three Django-based microservices handle all business logic, each isolated in a private subnet with no public IP:

- Data Collection Service (private-1, eu-central-1a, 10.0.128.0/20) — Receives drone imagery and flight metadata. Writes uploaded image files to the vine-care-bucket S3 bucket for durable object storage.
- Notification Service (private-3, eu-central-1a, 10.0.160.0/20) — Sends alerts and recommendations to farm managers when phenology events or KPI thresholds are triggered.

- Phenology Service (private-4, eu-central-1b, 10.0.176.0/20) — Tracks vine growth stages throughout the growing season. Placed in a different AZ from the other two backend services for additional fault tolerance.

6. Load Balancing

Three Application Load Balancers distribute incoming HTTP traffic across the frontend instances. All are internet-facing, span two Availability Zones, and reside within vine-care-vpc.



Screenshot 4 — AWS EC2 Console: Load Balancers

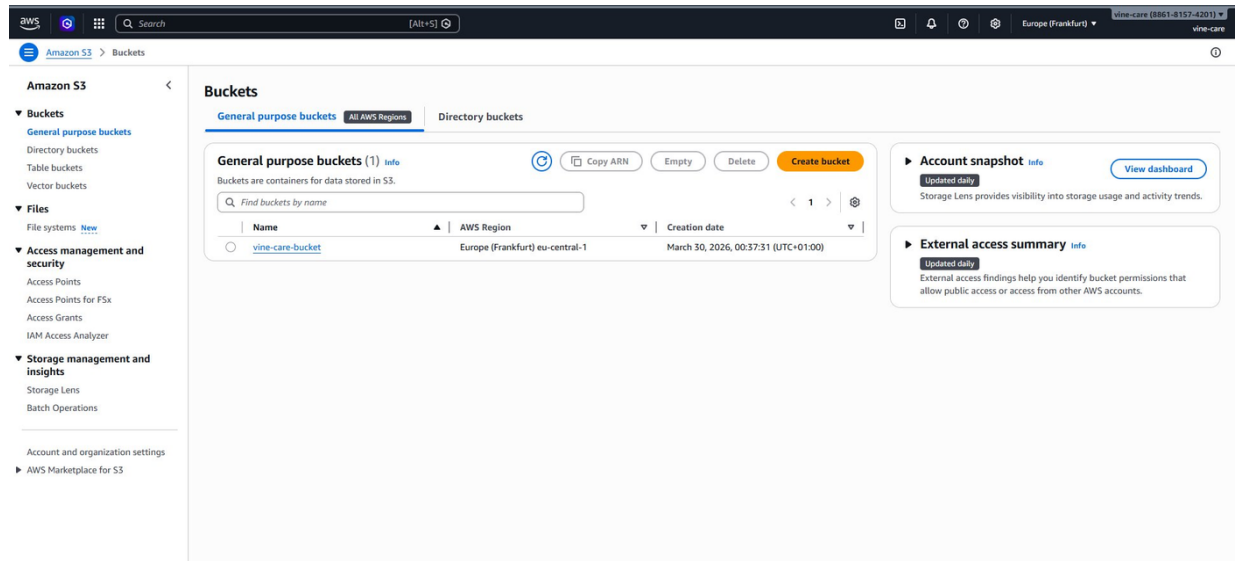
Name	Type	Scheme	IP Mode	Availability
vine-care-lb	Application	Internet-facing	IPv4	2 AZs — Active
vine-care-frontend-alb	Application	Internet-facing	Dualstack	2 AZs — Active
vine-care-frontend-alb-2	Application	Internet-facing	IPv4	2 AZs — Active

Table 3 — Application Load Balancer Configuration

vine-care-frontend-alb is configured with Dualstack (IPv4 + IPv6) addressing for compatibility with modern IPv6 clients. The dual-ALB design gives each frontend instance its own dedicated load balancer, enabling independent scaling, independent health checking, and granular traffic routing.

7. Object Storage — Amazon S3

VineCare uses Amazon S3 for scalable, durable object storage. The single general purpose bucket vine-care-bucket was created to store all drone imagery uploaded from the field. Below is the AWS S3 console view.



Screenshot 5 — AWS S3 Console: vine-care-bucket

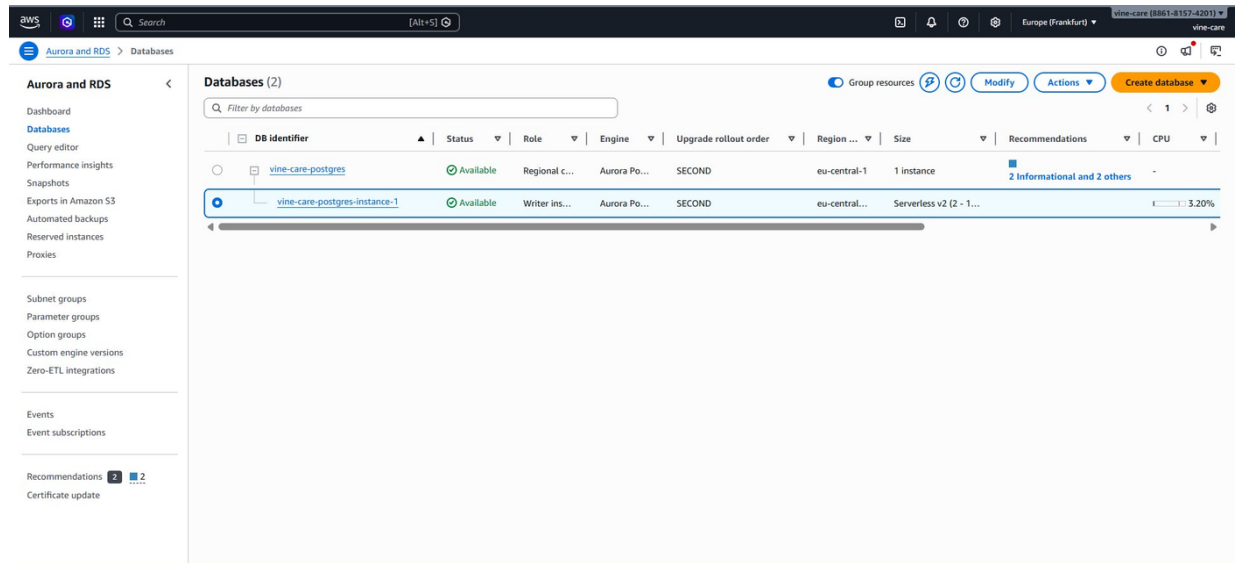
Property	Value
Bucket Name	vine-care-bucket
Bucket Type	General Purpose
AWS Region	Europe (Frankfurt) — eu-central-1
Purpose	Drone imagery storage (JPG/PNG/JPEG uploads from field operators)
Access	Private — accessed by backend EC2 instances via IAM roles (no public access)

Table 4 — S3 Bucket Configuration

When a field operator uploads drone images through the VineCare Upload interface, the files are received by the data-collection Django microservice running in a private subnet. That service then writes the image files to vine-care-bucket via the AWS SDK using an IAM role — no public S3 access is required or enabled. This approach decouples file storage from compute, allowing images to be stored durably and accessed independently of the EC2 instance lifecycle.

8. Database — Aurora PostgreSQL Serverless v2

Persistent structured data is stored in an Amazon Aurora PostgreSQL cluster named vine-care-postgres in the eu-central-1 region. Below is the AWS RDS console view.



Screenshot 6 — AWS Aurora & RDS Console: vine-care-postgres Cluster

Property	Value
Cluster Identifier	vine-care-postgres
Instance Identifier	vine-care-postgres-instance-1
Engine	Amazon Aurora PostgreSQL (SECOND upgrade rollout)
Instance Role	Writer (Primary)
Scaling	Serverless v2 — auto-scales ACUs with demand
Region	eu-central-1 (Frankfurt)
Subnets	RDS-Pvt-subnet-1 / RDS-Pvt-subnet-2 / RDS-Pvt-subnet-3
Failover	Multi-AZ with automatic failover
Status	☒ Available

Table 5 — Aurora PostgreSQL Database Configuration

Aurora PostgreSQL Serverless v2 is ideal for VineCare because drone upload events are bursty and seasonal. The cluster automatically scales Aurora Capacity Units (ACUs) upward during heavy image upload or analysis periods, and scales back down during off-season or quiet hours — optimising cost without sacrificing performance. Multi-AZ deployment ensures automatic failover with no manual intervention.

9. Application Modules

The VineCare web application is accessed through the ALB DNS endpoint. It is a full-featured precision agriculture platform with the following modules:

Module	Description
 Farms & Blocks	Vineyard blocks defined by GPS boundary polygons (4–5 coordinate points). Each block tagged with grape variety, farm name, and location. Varieties: Gewurztraminer, Assyrtiko, Merlot, Chardonnay, Cabernet Sauvignon.
 Drone Flights	Operators upload JPG/PNG/JPEG drone images (up to 100 per batch) with block ID and flight altitude. Uploaded files are stored in vine-care-bucket (S3) and processed by the data-collection microservice.
 Phenology Tracking	Dedicated service tracks vine growth stages through the growing season to support evidence-based farm interventions.
 KPIs	Agronomic key performance indicators surfaced through the frontend dashboard.
 Actions	Farm action logging and task management for field crews, linked to block-level observations.
 Predictions	ML-based yield and disease risk predictions displayed to farm managers.
 Notifications	Dedicated notification microservice sends alerts and recommendations triggered by phenology events or KPI thresholds.

Table 6 — Application Module Summary

9.1 Upload Interface

Field operators access the Upload module to submit drone imagery. Images are tagged with a Block ID and flight altitude (meters). Up to 100 images can be uploaded per batch (JPG, PNG, JPEG). Files are stored in vine-care-bucket (S3) via the data-collection service.

Upload Images

BLOCK: e.g. A1

METERS: e.g. 150

Drag & drop images or click to browse
JPG, PNG, JPEG - Max 100 images

Upload

Screenshot 7 — VineCare Application: Upload Images Interface

9.2 Blocks View

The Blocks module shows all vineyard blocks registered for KOKOTOS ESTATE. Each block displays its grape variety, farm name, and precise GPS boundary coordinates (4–5 polygon points). Managers can filter by farm using the dropdown.

Blocks (5) All Farms

Block Name	Grape Variety	Farm	Boundary Points
Gewurztraminer – lower	Gewurztraminer	KOKOTOS ESTATE	4 points
Assyrtiko – lower	Assyrtiko	KOKOTOS ESTATE	4 points
Merlot – mid	Merlot	KOKOTOS ESTATE	4 points
Chardonnay – lower	Chardonnay	KOKOTOS ESTATE	5 points
Cabernet Sauvignon – upper	Cabernet Sauvignon	KOKOTOS ESTATE	4 points

Screenshot 8 — VineCare Application: Blocks Management View

10. Security & High Availability

10.1 Network Isolation

- Backend Django services operate exclusively in private subnets — no public IPs, unreachable from the internet.
- Only ALBs and frontend EC2 instances in public subnets accept inbound internet traffic.
- NAT Gateway allows private instances to initiate outbound connections while blocking all inbound connections from external sources.
- RDS subnets are dedicated private subnets with a separate route table (RDS-Pvt-rt), isolating all database traffic.
- S3 bucket (vine-care-bucket) has no public access. Backend services access it via IAM roles and the AWS SDK, keeping all data transfers internal to AWS.

10.2 Security Groups

- ALB Security Group — Permits inbound HTTPS (443) from the internet only.
- Frontend EC2 Security Groups — Accept inbound traffic only from the ALB security group.
- Backend EC2 Security Groups — Accept inbound traffic only from frontend instances. Fully private.
- RDS Security Group — Allows PostgreSQL (5432) from backend EC2 instances only. No external database access possible.

10.3 High Availability

- Frontend: Two instances across eu-central-1a and eu-central-1b. ALB automatically reroutes if one AZ fails.
- Backend: Three services spread across both AZs — two in eu-central-1a, one in eu-central-1b. No AZ failure takes down all microservices.
- S3: Automatically replicates objects across multiple AZs within eu-central-1 — 99.999999999% (11 nines) durability.
- Database: Aurora PostgreSQL Serverless v2 replicates across multiple AZs with automatic failover, typically under 30 seconds.
- Load Balancers: All three ALBs span 2 Availability Zones, eliminating single points of failure at the traffic entry layer.

11. Conclusion & Future Roadmap

The VineCare AWS architecture is designed for production reliability, operational simplicity, and cost-efficiency. Every design decision addresses a real precision agriculture requirement:

Design Decision	Benefit
Multi-AZ deployment	Resilience against single AZ failure — critical for always-on farm monitoring.
Private backend subnets	No backend service is directly exposed to the internet — strong security posture.
S3 for image storage	Decouples file storage from compute; 11-nines durability; scales to millions of drone images.
Aurora Serverless v2	Auto-scales with bursty drone upload seasons; cost-efficient in off-season.
Separate ALBs per frontend	Independent scaling, health checking, and traffic control per frontend instance.
NAT Gateway	Backend services can update and call external APIs securely without public IPs.
t3.micro instances	Burstable compute — handles typical farm API traffic with cost efficiency.

Future Enhancement Opportunities

- Auto Scaling Groups — Automatically add/remove EC2 instances based on CPU or request metrics during peak drone upload seasons.
- Amazon CloudFront — CDN layer in front of the ALBs for global low-latency access and DDoS protection.
- AWS WAF — Web Application Firewall on the ALBs to block malicious requests.
- AWS Secrets Manager — Centralised, rotated database credentials management for all backend services.
- S3 Lifecycle Policies — Automatically move older drone images to S3 Glacier for long-term low-cost archival.
- Amazon SES / SNS — Managed email and SMS delivery for the notification microservice.